

BANCO DE DESARROLLO PRODUCTIVO — BDP

S.A.M.

Sistema de Análisis de Microcrédito

DOCUMENTACIÓN TÉCNICA DE ARQUITECTURA

Arquitectura Hexagonal · CQRS · Event Sourcing · Axon Framework

Universidad Adventista de Bolivia (UAB)

Ingeniería de Sistemas — Ingeniería de Software

Sprint 1 – Sprint 2 | 2026

1. VISIÓN GENERAL Y FUNDAMENTOS ARQUITECTÓNICOS

El sistema no es un "CRUD" tradicional (Crear, Leer, Actualizar, Borrar). Se construyó para cumplir normativas bancarias de la ASFI donde la auditoría y la precisión financiera son innegociables.

¿Por qué elegimos esta arquitectura?

Paradigma	Problema que resuelve	Implementación
Arquitectura Hexagonal	Aísla la lógica de negocio de las tecnologías externas. Si el banco cambia de PostgreSQL a Oracle, o de React a Angular, el núcleo financiero no se toca.	Paquete core/ independiente de frameworks. Puertos de entrada en adapters/in/web/.
CQRS	Separa las operaciones de escritura (lentas, con validaciones estrictas) de las de lectura (ultrarrápidas, optimizadas para consultas).	Command Side: *Command + *Aggregate. Query Side: Controladores y Proyectores.
Event Sourcing (Axon Framework)	En un banco no se puede hacer un UPDATE y borrar el saldo anterior. Cada cambio se guarda como un evento inmutable. Si hay error, se puede 'rebobinar' el estado a cualquier punto.	Axon Server almacena eventos. @EventSourcingHandler reconstruye el estado del agregado.

2. ESTRUCTURA DEL MONORREPOSITORIO

El proyecto utiliza un Monorepo para facilitar la Integración Continua y mantener sincronizadas las versiones del Frontend y Backend.

```
C:\BDP-SAM-Project\  
├── backend/           # Aplicación Spring Boot (Java)  
├── web-dashboard/     # Aplicación React (TypeScript)  
├── infra/             # Infraestructura como Código (Docker)  
├── docs/              # Documentación técnica  
└── mobile/           # Estructura de App Android (Futuro)
```

2.1. Archivos Clave de Configuración

Archivo	Ubicación	Propósito
pom.xml	backend/	Gestor de dependencias Maven (Spring Boot 3.5.13, Axon 4.9.0, PostgreSQL, H2, Lombok).
application.properties	backend/src/main/resources/	Puerto 8080, conexión a Axon Server (localhost:8124), H2 en memoria para desarrollo.
package.json	web-dashboard/	Gestor Node (React 18, MUI 9, Axios, jsPDF, Vite).

Archivo	Ubicación	Propósito
vite.config.ts	web-dashboard/	Configuración del empaquetador Vite (compila el frontend en milisegundos).
docker-compose.yml	infra/	Orquesta 4 contenedores: PostgreSQL, Axon Server, WireMock y pgAdmin.

3. BACKEND: EL CEREBRO BANCARIO — ANATOMÍA DEL FLUJO CQRS

3.1. Estructura de Paquetes Principal

```

bo.gob.bdp.sam
├── bdp_credit_backend/
│   ├── BdpCreditBackendApplication.java  # @SpringBootApplication
│   └── config/
│       └── CorsConfig.java                # Tráfico HTTP desde puerto 5173
├── core/
│   ├── application/
│   │   ├── command/                      # Comandos CQRS (escritura)
│   │   └── query/                        # Consultas CQRS (lectura)
│   ├── domain/
│   │   ├── aggregate/                   # Lógica de negocio
│   │   ├── event/                       # Eventos de Dominio (Inmutables)
│   │   └── exception/                   # Excepciones personalizadas
├── adapters/
│   ├── in/
│   └── web/                             # Controladores REST

```

3.2. Flujo de Trabajo CQRS (Paso a Paso)

Paso	Descripción
1. Controlador REST	Recibe petición HTTP del frontend (Axios). Ejemplo: ClienteController → POST /api/v1/clientes/registrar. Crea el comando y lo envía al CommandGateway.
2. Comando	Objeto inmutable que representa la INTENCIÓN de acción. Ejemplo: RegistrarClienteCommand(clienteId, nombre, email, telefono). Anotación clave: @TargetAggregateIdentifier.
3. CommandGateway	Enruta el comando al Agregado correcto. El controlador NO sabe qué agregado lo procesará.
4. Agregado	Contiene la LÓGICA DE NEGOCIO (@CommandHandler). Valida reglas: documento mínimo 7 dígitos, nombre no vacío, etc. Si todo es válido → aplica un Evento.
5. Evento	Objeto INMUTABLE que registra el hecho ocurrido. Ejemplo: ClienteRegistradoEvent. Usa @Value de Lombok (todos los campos son final).
6. Axon Server	Almacena el evento de forma inmutable (Auditoría ASFI). Puerto 8124 (gRPC) para comandos/eventos. Puerto 8024 para interfaz web de monitoreo.

Paso	Descripción
7. @EventSourcingHandler	Reconstruye el estado del agregado desde los eventos almacenados. Aquí SOLO se asignan variables, NUNCA se valida.
8. Proyector	Clase con @Component y @EventHandler. Escucha el evento y guarda los datos en PostgreSQL usando Spring Data JPA.

3.3. Los Comandos (La Intención del Usuario)

Un comando es una clase inmutable (con @Data de Lombok y campos final). Representa una orden. La anotación clave @TargetAggregateIdentifier le dice a Axon a qué agregado específico enrutar el comando.

```
@Data
@AllArgsConstructor
public class ActualizarBalanceCommand {
    @TargetAggregateIdentifier
    private final String clienteId;
    private final Map<String, Double> cuentas;
    private final boolean validacionFinal; // true = bloquear si descuadrado
}
```

3.4. El Agregado (El Guardia de Seguridad)

Es la clase más importante del dominio. Vive en core/domain/aggregate/. No se conecta a la base de datos; solo aplica matemáticas y reglas de negocio en memoria RAM.

Anotación	Propósito
@Aggregate	Le dice a Spring que esta clase maneja el ciclo de vida de una entidad bancaria.
@AggregateIdentifier	Define la llave primaria (ID).
@CommandHandler	Método que 'atrapa' el Comando. Aquí van los if/else de validación.
@EventSourcingHandler	Método que reconstruye el estado desde eventos. Aquí solo se asignan variables, no se valida.

Ejemplo real — Volteo de Balances (RF-03.1):

```
@CommandHandler
public BalanceAggregate(ActualizarBalanceCommand command) {
    // 1. Validar que el cliente existe
    if (command.getClienteId() == null || command.getClienteId().isEmpty()) {
        throw new ClienteNoEncontradoException("Cliente no encontrado");
    }
    // 2. Calcular totales con BigDecimal (NO double)
    BigDecimal activoTotal = activoCorriente.add(activoNoCorriente);
    BigDecimal pasivoPatrimonio = pasivoTotal.add(patrimonio);
    BigDecimal diferencia = activoTotal.subtract(pasivoPatrimonio).abs();
    // 3. Validar RNF-02 (±10 Bs)
    BigDecimal margen = new BigDecimal("10.00");
```

```

boolean cuadrado = diferencia.compareTo(margen) <= 0;
// 4. Si validacionFinal y descuadrado → BLOQUEAR
if (command.isValidacionFinal() && !cuadrado) {
    throw new BalanceDescuadradoException(
        "Balance descuadrado. Diferencia: " + diferencia + " Bs");
}
// 5. Si todo bien → APLICAR EVENTO
AggregateLifecycle.apply(new BalanceActualizadoEvent(...));
}

```

💡 ¿Por qué BigDecimal y no double?

En montos grandes, double agrega decimales basura (ej. 10.0000000000001). BigDecimal con .compareTo() garantiza matemática bancaria perfecta, cumpliendo el RNF-02 de precisión financiera.

3.5. Los Eventos (Lo Que Realmente Sucedió)

Son objetos inmutables que registran un hecho ocurrido. Usan @Value de Lombok (todos los campos son final). Una vez creado, un evento jamás se modifica. Es la fuente de verdad para la auditoría ASFI.

```

@Value
public class BalanceActualizadoEvent {
    private final String clienteId;
    private final Map<String, Double> cuentas;
    private final boolean balanceCuadrado;
    private final double diferencia;
    private final String timestamp;
}

```

3.6. Los Proyectores (La Escritura en PostgreSQL)

¿Cómo llega el dato a la base de datos si el Agregado no sabe de SQL? A través de los Proyectores (Projections). Anotados con @Component y @EventHandler, escuchan los eventos de forma asíncrona y usan repositorios Spring Data JPA para persistir en PostgreSQL. Cuando el frontend pide información, hace un SELECT rápido directo a esta tabla, sin pasar por el Agregado.

3.7. Catálogo Completo de Módulos Implementados

RF	Comando	Agregado	Evento	Controlador
RF- AU- TH- 01	CargarModelosCommand	ModeloEvaluacion Aggregate	ModelosCargadosEven t	ModeloController
RF- 02 .1	RegistrarClienteComm and	ClienteAggregate	ClienteRegistradoEv ent	ClienteController

RF	Comando	Agregado	Evento	Controlador
RF-02.2	ActualizarChecklistCommand	DocumentoAggregate	ChecklistActualizadoEvent	DocumentoController
RF-03.1	ActualizarBalanceCommand	BalanceAggregate	BalanceActualizadoEvent	BalanceController
RF-03.2	CalcularIndicadoresCommand	IndicadorAggregate	IndicadoresCalculadosEvent	IndicadorController
RF-03.3	GenerarPlanPagoCommand	SimuladorPagoAggregate	PlanPagoGeneradoEvent	SimuladorPagoController
RF-04.1	RegistrarEvaluacionAgricolaCommand	EvaluacionAgricolaAggregate	EvaluacionAgricolaRegistradaEvent	EvaluacionAgricolaController
RF-05.1	RegistrarEvaluacionPecuariaCommand	EvaluacionPecuariaAggregate	EvaluacionPecuariaRegistradaEvent	EvaluacionPecuariaController
RF-06.1	RegistrarEvaluacionProduccionCommand	EvaluacionProduccionAggregate	EvaluacionProduccionRegistradaEvent	EvaluacionProduccionController

3.8. Excepciones de Dominio Personalizadas

Excepción	Ubicación	Propósito
UsuarioNoAutenticadoException	core/domain/exception/	Usuario no autenticado o ID vacío.
DocumentoInvalidoException	core/domain/exception/	CI/NIT con formato incorrecto (< 7 dígitos).
ClienteNoEncontradoException	core/domain/exception/	Cliente nulo o inexistente.
DocumentacionIncompletaException	core/domain/exception/	Checklist con documentos obligatorios faltantes.
BalanceDescuadradoException	core/domain/exception/	Descuadre contable superior a ±10 Bs (RNF-02).

4. FRONTEND: REACT, TYPESCRIPT Y EL FLUJO DE ESTADOS

4.1. ¿Por qué TypeScript y no JavaScript puro?

Ventaja de TypeScript	Ejemplo en nuestro proyecto
Tipado estático	Si un componente espera un number y recibe un string, TypeScript lo detecta en VSCode antes de compilar.
Interfaces	Definimos interface IndicadoresData para garantizar que el objeto recibido tenga liquidezCorriente, endeudamiento, etc.
Autocompletado	Al escribir resultado., el editor sugiere liquidezCorriente, endeudamiento, etc.

4.2. Los Hooks de React (El Corazón del Frontend)

Hook	Propósito	Ejemplo en el proyecto
useState	Crea variables que, cuando cambian, actualizan la pantalla automáticamente.	En RegistroCliente.tsx, cada vez que el usuario teclea su Carnet, un useState guarda ese número.
useEffect	Ejecuta código apenas la pantalla se abre.	En IndicadoresFinancieros.tsx, un useEffect llama a Axios para buscar los datos del backend.
useParams	Extrae el ID de la URL.	Si la URL es /checklist/12345, este hook captura 12345 para saber a qué cliente pertenecen los documentos.
useNavigate	Redirige a otra página programáticamente.	Después de registrar un cliente, navigate('/checklist/1234567') lo lleva al checklist.

4.3. Comunicación con el Backend (Axios)

Se usó la librería Axios para las llamadas HTTP. Todo está envuelto en un bloque try/catch. Si el backend responde con un Error 400 (ej. balance descuadrado), el bloque catch activa un componente MUI Snackbar para mostrar una notificación roja en la esquina de la pantalla.

```
const handleSubmit = async () => {
  try {
    const response = await axios.post(
      'http://localhost:8080/api/v1/clientes/registrar', formData);
    // Éxito → Snackbar verde → Redirigir
  } catch (err: any) {
    // Error 400 (validación) → Snackbar rojo con mensaje del backend
    // Error 500 (servidor) → Mensaje genérico
    // Error de red (timeout) → "Error de conexión con el servidor"
  }
};
```

4.4. Rutas del Frontend (App.tsx)

Ruta	Componente	RF
/	MenuPrincipal	RF-AUTH-01
/registro-cliente	RegistroCliente	RF-02.1
/checklist/:clienteId	ChecklistDocumentos	RF-02.2
/volteo-balances/:clienteId	VolteoBalances	RF-03.1
/indicadores/:clienteId	IndicadoresFinancieros	RF-03.2
/simulador-pagos/:clienteId	SimuladorPagos	RF-03.3
/evaluacion-agricola/:clienteId	EvaluacionAgricola	RF-04.1
/evaluacion-pecuaria/:clienteId	EvaluacionPecuaria	RF-05.1
/evaluacion-produccion/:clienteId	EvaluacionProduccion	RF-06.1

5. INFRAESTRUCTURA DEVOPS (LA ORQUESTACIÓN DOCKER)

5.1. El docker-compose.yml Desglosado

Este archivo levanta 4 contenedores que se comunican a través de una red interna cerrada:

Servicio	Imagen	Puerto	Función	Detalle Técnico
axonserver	axoniq/axonserver:latest	8024, 8124	Cerebro del Event Sourcing	Puerto 8124 usa gRPC. Puerto 8024 es la interfaz web de monitoreo.
postgres_bdp	postgres:15-alpine	5432	Base de datos relacional	Configurado con variables de entorno de seguridad. Volumen mapeado para persistencia.
pgadmin	dpape/pgadmin4:latest	5050	Administración visual	Acceso por navegador para gestionar tablas gráficamente sin CLI.
core_simulator	wiremock/wiremock:latest	8081	Simulador CORE Argosnet	Actúa como 'impostor': recibe peticiones y

Servicio	Imagen	Puerto	Función	Detalle Técnico
				devuelve JSON preprogramado.

Comandos de operación:

```
cd infra
docker compose up -d      # Levantar todos los servicios
docker ps                 # Verificar contenedores corriendo
docker compose down       # Detener todo
```

6. ANATOMÍA EXACTA DE UNA PETICIÓN HTTP (MÓDULO BALANCES)

6.1. El Frontend Envía la Petición

URL: POST <http://localhost:8080/api/v1/balances/CLI-998/validar>

JSON Enviado por React (Payload):

```
{
  "activoCorriente": 50000.00,
  "activoNoCorriente": 100000.00,
  "pasivoCorriente": 20000.00,
  "pasivoNoCorriente": 30000.00,
  "patrimonio": 99995.00
}
```

6.2. El Backend Procesa la Petición

- BalanceController recibe el JSON.
- Crea el comando ActualizarBalanceCommand con validacionFinal = true.
- CommandGateway enruta el comando a BalanceAggregate.
- Cálculo: Activo Total = 50000 + 100000 = 150000
- Pasivo + Patrimonio = 20000 + 30000 + 99995 = 149995
- Descuadre = |150000 - 149995| = 5 Bs → $5 \leq 10$ → APROBADO
- Se aplica el evento BalanceActualizadoEvent y Axon Server lo almacena.
- El Proyector escucha el evento y guarda el registro en PostgreSQL.

6.3. El Backend Responde al Frontend

JSON Devuelto (HTTP 200 OK):

```
{
  "estado": "EXITO",
  "mensaje": "Balance validado. Descuadre dentro del margen permitido (5.00 Bs).",
  "fechaValidacion": "2026-06-14T04:20:00Z"
}
```

6.4. Si el Balance Estuviera Descuadrado (>10 Bs)

JSON Devuelto (HTTP 400 Bad Request):

```
{
  "error": "BALANCE RECHAZADO",
  "detalle": "Descuadre de 2000.00 Bs supera el margen permitido de ±10 Bs.",
  "validacion": "RECHAZADA"
}
```

7. BITÁCORA TÉCNICA: PROBLEMAS REALES RESUELTOS

Los siguientes hitos de ingeniería representan los principales desafíos técnicos enfrentados y resueltos durante el desarrollo:

7.1. El Problema del CORS (Cross-Origin Resource Sharing)

Síntoma: React intentaba mandar datos a Java, pero el navegador bloqueaba la petición por seguridad (React en puerto 5173, Java en 8080).

Solución: Se creó la clase CorsConfig.java implementando WebMvcConfigurer:

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
            .allowedOrigins("http://localhost:5173")
            .allowedMethods("GET", "POST", "PUT", "DELETE");
    }
}
```

7.2. Pérdida de Precisión Financiera (double vs BigDecimal)

Síntoma: Los balances usaban double. En montos grandes, Java agregaba decimales basura (ej. 10.000000000001), haciendo fallar la validación exacta de 10 Bs.

Solución: Se refactorizó toda la capa de Dominio para usar BigDecimal y métodos .compareTo(), garantizando matemática bancaria perfecta.

7.3. El Mock del CommandGateway (LocalAxonConfig.java)

Síntoma: Las peticiones POST no funcionaban porque LocalAxonConfig.java creaba un proxy falso del CommandGateway que no se conectaba a nada.

Solución: Se eliminó el archivo y se dejó que Spring inyectara el CommandGateway real de Axon Framework, conectándose al Axon Server en Docker.

7.4. El Problema de Sincronía del CQRS

Síntoma: CQRS es 'Eventualmente Consistente'. Entre que el Agregado acepta el comando y el Proyector guarda el dato en PostgreSQL pasan milisegundos. A veces el Frontend recargaba tan rápido que el dato aún no estaba en la base de datos.

Solución: Se manejó el asincronismo en los controladores usando `CompletableFuture<String>`, asegurando que Java no responda con HTTP 200 OK hasta que todo el flujo esté verdaderamente completo.

7.5. El Escaneo de Paquetes de Spring Boot

Síntoma: El backend respondía 404 en todas las rutas porque los controladores estaban en `bo.gob.bdp.sam.adapters.in.web`, pero la clase principal en `bo.gob.bdp.sam.bdp_credit_backend`. Spring solo escanea el paquete de la clase principal.

Solución: Se agregó `scanBasePackages = "bo.gob.bdp.sam"` en la anotación `@SpringBootApplication`.

7.6. La Vulnerabilidad de Bouncy Castle (CVE-2024-30172)

Síntoma: El análisis OWASP Dependency-Check detectó una vulnerabilidad en `bcprov-jdk15on:1.70`.

Solución: Se actualizó a `bcprov-jdk18on:1.79`, mitigando la vulnerabilidad y cumpliendo con las políticas de seguridad ASFI.

8. API REST: CATÁLOGO COMPLETO DE ENDPOINTS

Módulo	Método	Endpoint	Descripción
Modelos	GET	<code>/api/v1/modelos/menu</code>	Lista los 3 modelos de evaluación.
	POST	<code>/api/v1/modelos/cargar</code>	Registra intención de carga (auditoría).
Clientes	POST	<code>/api/v1/clientes/registrar</code>	Crea un nuevo expediente de cliente.
Documentos	GET	<code>/api/v1/documentos/checklist/{id}</code>	Obtiene estado del checklist.
	POST	<code>/api/v1/documentos/checklist/{id}</code>	Actualiza el checklist.
Balances	GET	<code>/api/v1/balances/{id}</code>	Obtiene balance y validación ± 10 Bs.
	POST	<code>/api/v1/balances/{id}</code>	Guarda balance (borrador).
	POST	<code>/api/v1/balances/{id}/validar</code>	Valida balance final (bloquea si descuadre > 10 Bs).
Indicadores	GET	<code>/api/v1/indicadores/{id}</code>	Calcula liquidez, endeudamiento, ROA, ROE.

Módulo	Método	Endpoint	Descripción
Simulador	POST	/api/v1/simulador-pagos/generar	Genera plan de pagos con cronograma.
Agrícola	POST	/api/v1/evaluacion-agricola/registrar	Registra evaluación agrícola.
	GET	/api/v1/evaluacion-agricola/{id}	Consulta evaluación agrícola.
Pecuario	POST	/api/v1/evaluacion-pecuaria/registrar	Registra evaluación pecuaria.
	GET	/api/v1/evaluacion-pecuaria/{id}	Consulta evaluación pecuaria.
Producción	POST	/api/v1/evaluacion-produccion/registrar	Registra evaluación de producción.
	GET	/api/v1/evaluacion-produccion/{id}	Consulta evaluación de producción.

9. DOCUMENTACIÓN (docs/)

#	Archivo	Contenido	Estado
1	01_Introduccion_y_Contexto.md	Visión general, objetivos y alcance.	☒ Completo
2	02_Requerimientos_Funcionales.md	Lista completa de 14 RFs con priorización MoSCoW.	☒ Completo
3	03_Requerimientos_No_Funcionales_DevOps.md	7 RNFs de seguridad, rendimiento, disponibilidad.	☒ Completo
4	04_Mockups.md	Descripción de interfaces gráficas.	☒ Completo
5	05_Arquitectura.md	Arquitectura Hexagonal, CQRS, Event Sourcing.	☒ Completo
6	06_Diagramas/	5 diagramas UML (Casos de Uso, Actividades, Estados, etc.).	☒ Completo
7	07_Cronograma.jpeg	Hitos y fases del proyecto.	☒ Completo
8	08_Modelo_Entidad_Relacion.md	Tablas, atributos y relaciones.	☐ Pendiente
9	09_Plan_Capacitacion.md	Plan de capacitación para el BDP.	☐ Pendiente
10	Manual_Usuario.md	Guía para operadores finales.	☒ Completo
11	Manual_Tecnico_Despliegue.md	Guía para el equipo de IT del banco.	☒ Completo

